Programação funcional

Aula 02 - Cálculo λ*

*Baseado no material disponível em https://haskell.pesquisa.ufabc.edu.br/haskell

Cálculo λ

- Sistema formal para expressar computação baseado em abstração de funções e aplicação usando apenas atribuições de nomes e substituições .
  - Criado por Alonzo Church na década de 1930
  - Church foi o orientador de doutorado do Alan Turing

Menor linguagem universal?

- Nas linguagens que vocês aprenderam até agora, temos:
  - Atribuição
  - Booleanos, inteiros, floats, caracteres,…
  - Condicionais
  - Laços
  - Funções
  - Recursão
  - Ponteiros
  - Objetos, classes

Menor linguagem universal?

- Cálculo λ:
  - ~~Atribuição~~
  - ~~Booleanos, inteiros, floats, caracteres,…~~
  - ~~Condicionais~~
  - ~~Laços~~
  - Funções
  - ~~Recursão~~
  - ~~Ponteiros~~
  - ~~Objetos, classes~~

Sintaxe do cálculo λ

- 3 elementos
  - variáveis
  - definição de funções
  - aplicação de funções
- Em sua sintaxe original a letra grega minúscula lambda (λ) é usada para definir funções. Por exemplo:
  - Função identidade: λx.x
  - Função que devolve a função identidade: λx.(λy.y)
- Em Haskell, trocamos
  - **λ** por \ e **.** por ->
    - \x -> e

Sintaxe do cálculo λ

- 3 elementos:
  - Variável: x, y, z, um nome que assumirá um valor durante a computação.
  - Abstração: (função anônima ou função λ)
    - \x -> e
    - **x** é o parâmetro formal, **e** é o corpo da função
    - para qualquer valor **x** compute **e**
  - Aplicação: (ou chamada de função)
    - **e1 e2**, aplique o argumento e2 na função e1 (e1(e2))

Exercício

Qual dos seguintes termos está sintaticamente incorreto?

(a) \(\x -> x) -> y

(b) \x -> x x

(c) \x -> x (y x)

Exemplos

```
\x -> x              -- função identidade
\x -> (\y -> y)      -- retorna a função identidade
\f -> f (\x -> x)    -- função que aplica seu argumento
                     -- (que é uma função)
                     -- na função identidade
```

Funções que recebem mais de um argumento

```
\x -> (\y -> y) -- recebe dois args e retorna o segundo
\x -> (\y -> x) -- recebe dois args e retorna o primeiro
```

Açúcares sintáticos

original	syntatic sugar
`(((e1 e2) e3) e4)`	`e1 e2 e3 e4`
`\x -> (\y -> (\z -> e))`	`\x -> \y -> \z -> e`
`\x -> \y -> \z -> e`	`\x y z -> e`

Escopo de variáveis

```
int x; /* x está visível aqui, mas y não */
{
  int y;  /* x e y estão visíveis */
}
/* y deixou de existir :( */
```

Escopo de variáveis

- Na expressão \x -> e
  - **x** é uma variável
  - A expressão **e** é o escopo de **x**
  - Qualquer ocorrência de **x** em e está ligada (*bound*) por **\x**:

```
\x -> x
\x -> \y -> x
```

Escopo de variáveis

- **x** está livre (*free*) se não está dentro de uma abstração:

```
x y                 -- não tem \
\y -> x y           -- x vem de outro lugar
(\x -> \y -> x) x -- o segundo x é diferente do primeiro
```

- Se uma expressão **e** não tem variáveis livres, então **e** é uma expressão **fechada**

Semântica

- Para simplificar ou avaliar uma expressão lambda
  - Sequência de reduções até que nenhuma redução possa ser mais aplicada
- Podemos reescrever expressões utilizando 2 regras:
  - Passo α
    - renomeia um parâmetro formal.
  - Passo β
    - aplica uma expressão utilizando uma variável livre (ou, de maneira mais simples, chama uma função).

Semântica

- Redução β

```
(\x -> e1) e2 => e1[x := e2]
```

  - Onde e1[x := e2] significa, “e1 com todas as ocorrências livres de x substituídas por e2.”
  - Computação por substituição e busca
  - Se você vir uma abstração aplicada a um argumento, pegue o corpo da abstração e substitua todas as ocorrências livres do parâmetro formal pelo argumento

Forma normal

- Um termo-λ na forma (\x -> e1) e2 é chamado de expressão redutível (*reducible expression*) ou **redex**
- O termo está em sua forma normal se não contém nenhum **redex**.

Exercício

- Quais dos termos abaixo não está na forma normal?
  a. x
  b. x y
  c. (\x -> x) y
  d. x (\y -> y)

Exercício

- Quais dos termos abaixo não está na forma normal?
  a. x
  b. x y
  c. (\x -> x) y
  d. x (\y -> y)

Avaliação de expressões λ

- Um termo-λ **e** é avaliado para **e'** se existe uma sequência de passos (reduções α e/ou β):
  - e => e1 => ... => en => e'
- … tal que **e'** seja uma forma normal.

```
-- Exemplo 1
(\x -> x) 3 => 3

--Exemplo 2
(\f -> f (\x -> x)) (\x -> x)
    => (\x -> x) (\x -> x)
    => (\x -> x)
```

Non-Terminating Evaluation

- Considerando a expressão: (\x -> x x) (\x -> x x)
- Aplicando a redução β: (\x -> x x) (\x -> x x)
  - Ops… Conseguimos escrever programas que a cada redução nos leva a eles próprios
  - É impossível chegar a uma forma normal.
  - Este é o combinador Ω (*Ω combinator*)

Programando com λ

Programando com λ

- Linguagens reais tem muitas funcionalidades:
  - Booleanos
  - Registros (structs, tuplas, …)
  - Números
  - Funções
  - Recursão
- Até agora, temos apenas funções (λ)
- Vamos ver como representar o resto dessas funcionalidades com cálculo λ!

Booleanos

- Como fazer booleanos com λ

```
Verdadeiro = \x y -> x
Falso      = \x y -> y
IF         = \b x y -> b x y
```

- Verificando:

```
IF Verdadeiro 2 3
  => (\b x y -> b x y) Verdadeiro 2 3
  => (\x y -> Verdadeiro x y) 2 3
  => (\y -> Verdadeiro 2 y) 3
  => Verdadeiro 2 3
  => (\x y -> x) 2 3
  => (\y -> 2) 3
  => 2
```

Booleanos

- NOT, AND e OR

```
Verdadeiro = \x y -> x
Falso      = \x y -> y
IF         = \b x y -> b x y
```

```
NOT = \b -> IF b Falso Verdadeiro
AND = \b1 b2 -> IF b1 b2 Falso
OR  = \b1 b2 -> IF b1 Verdadeiro b2
```

Como IF é uma aplicação de função…

```
NOT = \b -> IF b Falso Verdadeiro
   => \b -> b Falso Verdadeiro

AND = \b1 b2 -> IF b1 b2 Falso
   => \b1 b2 -> b1 b2 Falso

OR  = \b1 b2 -> IF b1 Verdadeiro b2
   => \b1 b2 -> b1 Verdadeiro b2
```

Tuplas

```
PAIR = \x y -> (\b -> IF b x y)
FST  = \p -> p Verdadeiro -- Chamando com Verdadeiro, pega o primeiro elemento
SND  = \p -> p Falso      -- Chamando com Falso, pega o segundo elemento
```

```
TRIPLE = \x y z -> PAIR x (PAIR y z)
FST3   = \t -> FST t
SND3   = \t -> FST (SND t)
TRD3   = \t -> SND (SND t)
```

Números naturais

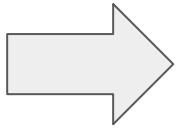
Números de Church: um número N é codificado como a chamada de uma função N vezes:

```
ZERO = ???
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```

Números naturais

Números de Church: um número N é codificado como a chamada de uma função N vezes:

```
ZERO = ???
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```



Aplicação de uma função 0 vezes

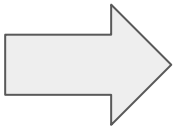
```
ZERO = \f x -> x
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```

Números naturais

Substituição α

```
ZERO  = \f x -> x
Falso = \x y -> y
```

```
ZERO = ???
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```



```
ZERO = \f x -> x
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```

Números naturais

```
ZERO = \f x -> x
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```

Função incremento:

```
INC = \n -> ???
-- Caso mais simples
INC ZERO = UM
INC (\f x -> x) = \f x -> f x
INC (\f x -> x) = \f x -> f ((\f x -> x) ???)
INC (\f x -> x) = \f x -> f ((\f x -> x) f x)
    β=> \f x -> f x = UM
-- Mas se você reparar
INC ZERO = \f x -> f (ZERO f x)
-- Assim, podemos fazer
INC = \n -> \f x -> f (n f x)
```

Redução β

Generalizando

Números naturais

Testando a função incremento:

```
INC = \n -> \f x -> f (n f x)
```

```
ZERO = \f x -> x
UM   = \f x -> f x
DOIS = \f x -> f (f x)
TRES = \f x -> f (f (f x))
```

```
INC UM -- Substituindo
(\n -> \f x -> f (n f x)) (\f x -> f x)
   β=> \f x -> f ((\f x -> f x) f x)
   β=> \f x -> f ((\x -> f x) x)
   β=> \f x -> f (f x)
   = DOIS

INC DOIS -- Substituindo
(\n -> \f x -> f (n f x)) (\f x -> f (f x))
   β=> \f x -> f ((\f x -> f (f x)) f x)
   β=> \f x -> f ((\x -> f (f x)) x)
   β=> \f x -> f (f (f x))
   = TRES
```

Números naturais

Soma:

```
    DOIS INC -- Substituindo
  = (\f x -> f (f x)) INC
β=> (\x -> INC (INC x)) = SOMA_DOIS
```

Aplicando a função DOIS à função INC

```
    QUATRO INC -- Substituindo
  = (\f x -> f (f (f (f x))) INC
β=> (\x -> INC (INC (INC (INC x))) = SOMA_QUATRO
```

Aplicando a função QUATRO à função INC

```
ADD = \n m -> n INC m
-- 'n INC' é função que incrementa x recebido como parâmetro n vezes
-- quando aplicada em m incrementa m, n vezes. Ou seja, resulta em m + n
```

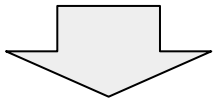
Generalizando

Recursão

Exemplo do fatorial

```haskell
fat x = if x == 0 then 1 else x * fat (x - 1)
-- Ou usando lambda
fat = \x -> if x == 0 then 1 else x * fat (x - 1)
```

No cálculo λ, as funções não tem nome



```haskell
fat = \f -> \x -> if x == 0 then 1 else x * f (x - 1)
-- Ou usando açúcar sintático
fat = \f x -> if x == 0 then 1 else x * f (x - 1)
```

```
fat = \f x -> if x == 0 then 1 else x * f (x - 1)
```

Recursão

Exemplo do fatorial

- Como calcular o fatorial de 3?
- O que passar no parâmetro f?
- fat ?? 3
  - Valor para f, p, tal que:
    - **p ≡ fat p**, ou em outras palavras, **p n ≡ fat p n**
  - Exemplo: **3! ≡ p 3 ≡ fat p 3 ≡ 6**
  - p é um ponto fixo* da função fat

*elemento do domínio que é mapeado para ele mesmo pela função, ex.: (0, 1, -1) para $f(x) = x^3$

Recursão

```
Ω ((\x -> x x) (\x -> x x))
```

Adaptação do combinador Ω:

```
A = (\x -> f (x x)) (\x -> f (x x))
```

- Além de infinitas reduções sem mudanças, incorpora a aplicação de uma função f:

```
A = (\x -> f (x x)) (\x -> f (x x))
  = f ((\x -> f (x x)) (\x -> f (x x)))
  = f A
```

Redução β

- **A = f A**, então A é ponto fixo de f

Recursão

Combinador Y:

- Expressão λ que devolve um ponto fixo para uma função λ

```
A = (\x -> f (x x)) (\x -> f (x x)) -- Definida acima
Y = \f -> A -- liga a variável f em A
Y = \f -> (\x -> f (x x)) (\x -> f (x x)) -- Combinador Y
```

Recursão

Fatorial com combinador Y

```
fat = \f x -> if x == 0 then 1 else x * f (x - 1)
Y = \f -> (\x -> f (x x)) (\x -> f (x x))
-- Quero calcular 3!
Y fat 3
= fat (Y fat) 3 -- Pois (Y fat) é ponto fixo de fat!
= (\f x -> if x == 0 then 1 else x * f (x - 1)) (Y fat) 3
= (\x -> if x == 0 then 1 else x * (Y fat) (x - 1)) 3
= if 3 == 0 then 1 else 3 * (Y fat) (3 - 1)
= 3 * (Y fat 2)
= 3 * (fat (Y fat) 2)
= 3 * ((\f x -> if x == 0 then 1 else x * f (x - 1)) (Y fat) 2)
= 3 * ((\x -> if x == 0 then 1 else x * (Y fat) (x - 1)) 2)
= 3 * (if 2 == 0 then 1 else 2 * (Y fat) (2 - 1))
= 3 * (2 * (Y fat) 1)
```

Recursão

Fatorial com combinador Y

```
= 3 * (2 * (Y fat) 1)
= 3 * 2 * ((Y fat) 1)
= 6 * (fat (Y fat) 1)
= 6 * ((\f x -> if x == 0 then 1 else x * f (x - 1)) (Y fat) 1)
= 6 * ((\x -> if x == 0 then 1 else x * (Y fat) (x - 1)) 1)
= 6 * (if 1 == 0 then 1 else 1 * (Y fat) (1 - 1))
= 6 * 1 * ((Y fat) 0)
= 6 * (fat (Y fat) 0)
= 6 * ((\f x -> if x == 0 then 1 else x * f (x - 1)) (Y fat) 0)
= 6 * ((\x -> if x == 0 then 1 else x * (Y fat) (x - 1)) 0)
= 6 * (if 0 == 0 then 1 else 0 * (Y fat) (0 - 1))
= 6 * 1
= 6
```

Referências

- Material do prof. Emilio Francesquini
  - https://haskell.pesquisa.ufabc.edu.br/haskell